Runbook — Bring a new Arabic model live behind Captain Adel

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-06-16

Captain Adel's brain (`captadel/`) ships with several Arabic / in-Kingdom model *options* — ALLaM (default), Jais, Fanar, Qwen, Command R — but **every one is OFF until you point it at a running endpoint.** With no endpoint set, the Arabic path simply isn't used and nothing changes. This runbook takes one Arabic model from "supported in code" to "answering real questions in production," safely.

It is the Arabic-slot companion to `runbook-captain-adel.md` (the gateway function) and `deploy/allam-vllm.md` in the captadel repo (serving ALLaM).

> **PDPL.** Real user questions are personal data and must be processed in-Kingdom. Host the model endpoint **and** the captadel service in a KSA region. Do not route Arabic traffic to a model served outside the Kingdom.

0. Decide which model

See `captadel/docs/models.md` for the full slot-B ranking. Short version:

- **ALLaM-7B** — default; best Saudi MSA + strongest sovereignty story. Start here.
- **Qwen2.5-14B/32B** — if ALLaM's instruction-following is too weak; best at obeying the cite-only contract; permissive licence.
- **Jais / Fanar** — strong Arabic-first alternatives to evaluate.
- **Command R** — best grounded-citation behaviour, **but CC-BY-NC**: evaluation only unless you hold a commercial licence. Do **not** ship it on the NC weights.

The steps below use `<name>` ∈ `allam | jais | fanar | qwen | commandr` and the matching env prefix `<PREFIX>` (e.g. `QWEN`).

1. Serve the model (vLLM, in-Kingdom GPU)

Any OpenAI-compatible `/chat/completions` server works (vLLM or TGI). With vLLM:

```
# On a KSA GPU box (A100/H100-class for 13B+; a 7B/9B fits a single 24-48 GB card)
pip install vllm
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen2.5-14B-Instruct \
  --served-model-name Qwen/Qwen2.5-14B-Instruct \
  --port 8000
# Endpoint is then  http://<host>:8000/v1
```

Put it behind TLS + an auth token if it's network-reachable. The served model name must match `<PREFIX>_MODEL` (the defaults are in `captadel/.env.example`).

---

2. Smoke-test the endpoint (≈2 s)

From `captadel/` on any machine that can reach the endpoint:

```
QWEN_BASE_URL=http://<host>:8000/v1  QWEN_API_KEY=<token-if-any> \
  node evals/provider-smoke.js qwen
```

Expect `PASS … response: "pong"`. If it fails it tells you why (wrong path/host, model-name mismatch, missing token). Fix before moving on.

---

3. Pass the parity gate (this is the real go/no-go)

Never route Arabic to a model on vibes. The gate runs every eval case through Gemini **and** the candidate and only passes if the candidate matches-or-beats Gemini on the **Arabic subset** without regressing overall:

```
GEMINI_API_KEY=<key>  QWEN_BASE_URL=http://<host>:8000/v1 \
  node evals/parity.js --provider qwen
```

- Exit 0 + `PARITY OK` → safe to route Arabic to this model.
- Exit 1 + `PARITY FAIL` → keep `MODEL_PROVIDER=gemini`; try another model (step 0) or improve retrieval (step 6) first.

Tip: `--arabic-only` focuses the run; `--tol 1` allows a 1-case overall regression.

---

4. Enable it on the captadel service

Set these on the **captadel.com** service (its host's env / secret manager — not the Firebase function):

```
QWEN_BASE_URL=http://<host>:8000/v1
QWEN_MODEL=Qwen/Qwen2.5-14B-Instruct      # optional; default in .env.example
QWEN_API_KEY=<token-if-any>               # optional
ARABIC_PROVIDER=qwen                      # make `auto` prefer this Arabic model
MODEL_PROVIDER=auto                       # Arabic-dominant → Arabic model, else Gemini
```

Leaving `ARABIC_PROVIDER` unset keeps ALLaM first in the preference order. The English/agentic path stays on Gemini regardless.

---

5. Deploy + verify

Deploy the captadel service (see `runbook-captadel-extraction.md` / `deploy/` for the container + KSA-region target), then:

```
# health
curl https://captadel.com/health      # → { status:"ok", ... }

# an Arabic turn should now answer in Arabic with Latin GACAR citations
curl -XPOST https://captadel.com/v1/chat -H 'Content-Type: application/json' \
  -d '{"message":"ما هو الحد الأدنى لسن رخصة الطيار الخاص؟","provider":"auto"}'
```

No front-end change is needed — `chat.html` / the gateway already speak the `{ message, history, product, provider, session }` contract. Roll back instantly by unsetting `<PREFIX>_BASE_URL` (or `MODEL_PROVIDER=gemini`).

---

6. Optional — hybrid retrieval (the cross-lingual unlock)

The GACAR corpus is essentially English, so an Arabic question retrieves little by lexical BM25 alone — an Arabic model will (correctly) refuse a lot. Dense, multilingual retrieval fixes this. It is **off by default**; turn it on only after building the index:

```
# 1. Serve an embeddings model (BGE-M3) the same OpenAI-compatible way, then
#    build the dense index once (writes captadel/src/brain/_embeddings.json.gz):
EMBEDDINGS_BASE_URL=http://<host>:8080/v1  npm run build:embeddings

# 2. Ship the generated _embeddings.json.gz with the service, and set at runtime:
EMBEDDINGS_BASE_URL=http://<host>:8080/v1     # BGE-M3
RERANK_BASE_URL=http://<host>:8081/v1         # optional: bge-reranker-v2-m3
```

With neither set, retrieval is pure BM25 and unchanged. Re-run the parity gate (step 3) with hybrid on — Arabic recall should jump, and more Arabic cases should pass.

Checklist

- ☐ Model served in a KSA region, behind TLS/auth.
- ☐ `provider-smoke.js <name>` → PASS.
- ☐ `parity.js --provider <name>` → PARITY OK (Arabic subset).
- ☐ `<PREFIX>_BASE_URL` (+ `ARABIC_PROVIDER`, `MODEL_PROVIDER=auto`) set on captadel.
- ☐ captadel deployed in a KSA region; `/health` OK; an Arabic turn answers in Arabic with Latin citations.
- ☐ (Optional) embeddings index built + `EMBEDDINGS_BASE_URL` set; parity re-checked.
- ☐ Rollback verified: unset `<PREFIX>_BASE_URL` returns to Gemini-only.